

NovaCart Account Dashboard: Solution Design Document

Team Tree

For all NovaCart employees

Version 1.0 | August 6, 2026

1. System Overview

NovaCart is a growing online retailer selling to customers in more than 30 countries. Until now, account managers have relied on ad-hoc Excel reports emailed weekly by the analytics team, often out of date and inconsistent. The NovaCart Account Dashboard replaces that process with a web based tool that lets account managers explore up to date business performance directly.

The dashboard connects directly to NovaCart's Snowflake data warehouse, reading order, customer, and product records that the Data Engineering team has already cleaned and validated. It only reads this information and never changes it; it is intended purely as a reporting and analysis tool for internal use.

1.1 What the Dashboard Shows

The dashboard presents business performance through three views:

- An orders overview, showing order volume and revenue trends over time.
- A product performance breakdown, showing which products generate the most revenue.
- A customer list, showing individual customers and their purchasing activity.

1.2 How the System is Organized

The dashboard is made up of a handful of cooperating parts. A React based visual interface presents information to the user. A routing layer (NGINX) directs incoming requests, sending anything related to data to the backend and everything else to the visual interface. A Python based API, built with FastAPI, performs calculations and retrieves figures on request, querying Snowflake's data tables directly using a secure, automatically issued credential rather than a stored password. The entire application runs as a set of containers on Snowpark Container Services (SPCS), a platform that operates inside Snowflake's own network. Figure 1 below shows how these parts connect and how a single request moves through the system.

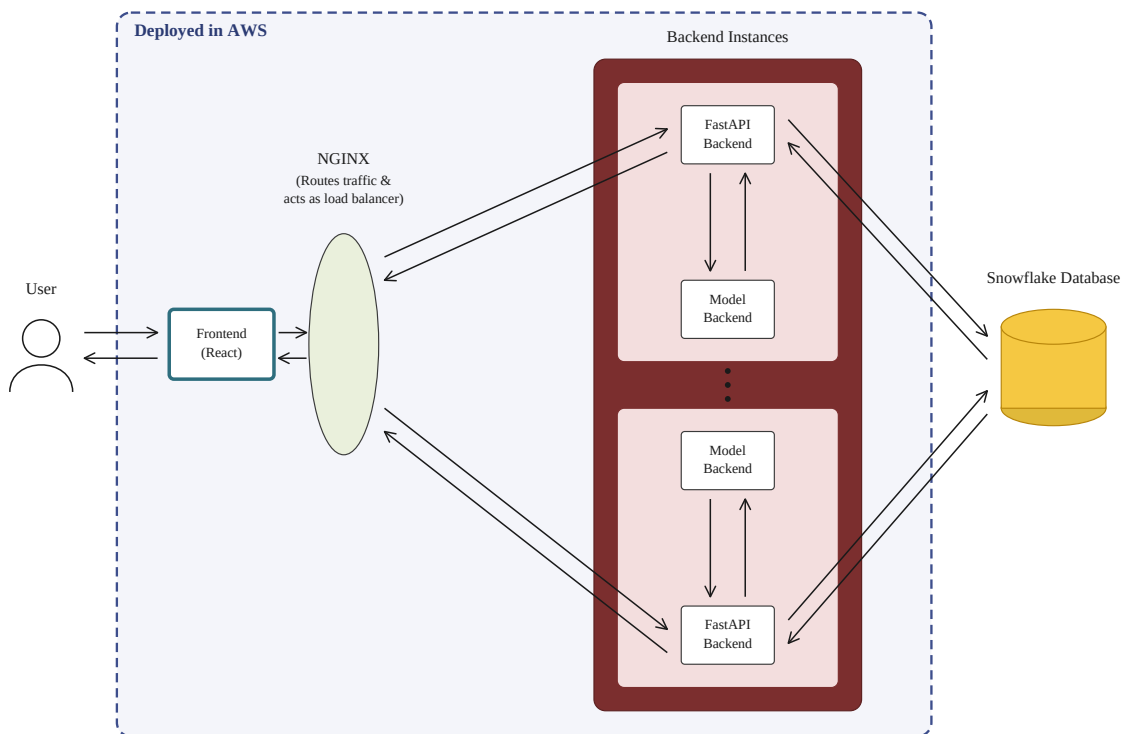


Figure 1. Request flow through the deployed system

The dashed boundary marks the portion of the system that runs as containers on the deployment platform; the user and the Snowflake database sit outside that boundary.

2. Key Technology Decisions

2.1 Centralized Request Routing (NGINX)

All incoming requests to the dashboard are directed through a single routing component before reaching any other part of the system. Requests for data are forwarded to the backend service; every other request is forwarded to the visual interface. This component also acts as a load balancer, distributing incoming traffic evenly if multiple copies of the backend are running.

Rationale

This approach provides a single, well-defined entry point for the system. It removes the need for complex cross-service permission configurations and simplifies the overall security posture.

Trade-off

This design introduces an additional processing step for every request. In addition, because only one routing component is currently deployed, its unavailability would render the entire application inaccessible, as no redundant instance is in place.

2.2 Lightweight Application Framework (FastAPI)

The component responsible for performing calculations and retrieving the figures displayed on the dashboard, such as revenue, order volume, and customer activity, is built in Python using FastAPI, a lightweight and widely adopted framework suited to this type of workload. It connects to Snowflake using a secure credential issued automatically by the platform, so no password or key is stored in the application itself.

Rationale

This framework enabled the development team to implement and modify these calculations efficiently, using established and well documented tools, without introducing unnecessary complexity for what are largely straightforward data queries. It also generates interactive documentation for every function automatically, so anyone reviewing the API can see exactly what it does without reading the underlying code.

Trade-off

To maintain simplicity, certain performance optimizations were not implemented, including connection reuse and caching of recent results. As a result, each request currently establishes a new connection to the data source, which may affect performance as the number of concurrent users increases.